

Neo4J Queries and Dataset

Part 1 – Neo4J CRUD movie database

Query 1 - bridges of madison county

So, the question asks to add the movie, three main actors, the director and the acted in relationship and the directed relationship. The movie being asked to include is the bridges of madison county, for this I used the MERGE instead OF CREATE to avoid creating duplicate movie or person nodes. Finally, this query adds 3 actors, the movie and the director. It also created ACTED_In relationship between the actors and the movie and a DIRECTED relationship between Clint Eastwood and the movie

```
MERGE (m:Movie {title: "The Bridges of Madison County"})
SET m.released = 1995,
    m.rating = 7.6

MERGE (clint:Person {name: "Clint Eastwood"})
SET clint.born = 1930

MERGE (meryl:Person {name: "Meryl Streep"})
SET meryl.born = 1949

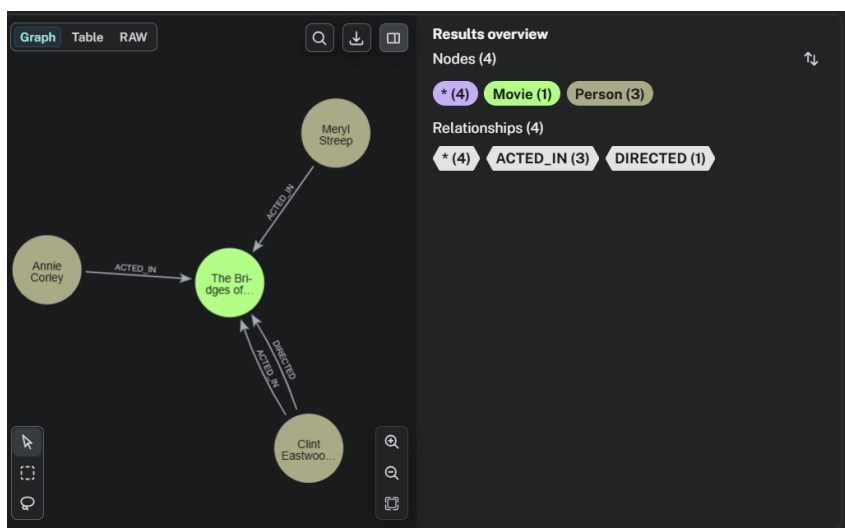
MERGE (annie:Person {name: "Annie Corley"})
SET annie.born = 1952

MERGE (clint)-[a1:ACTED_IN]->(m)
SET a1.roles = ["Robert Kincaid"]

MERGE (meryl)-[a2:ACTED_IN]->(m)
SET a2.roles = ["Francesca Johnson"]

MERGE (annie)-[a3:ACTED_IN]->(m)
SET a3.roles = ["Caroline Johnson"]

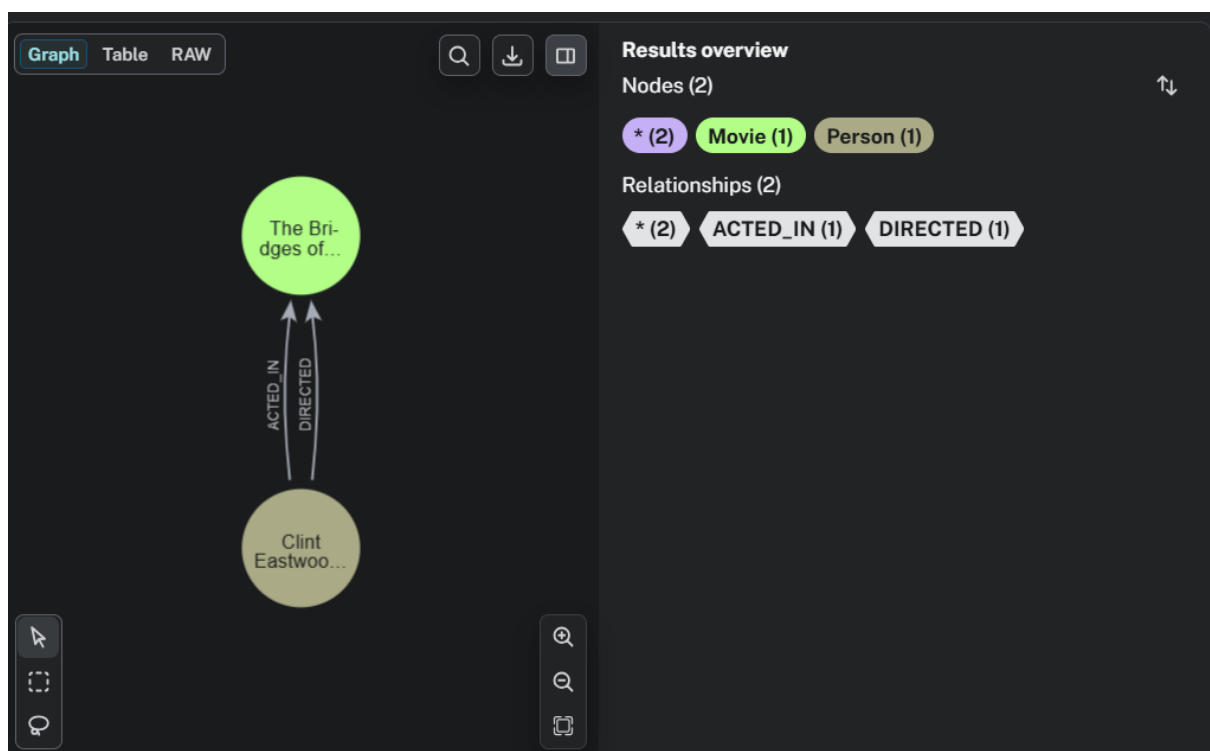
MERGE (clint)-[:DIRECTED]->(m);
```



Query 2 – actors who directed a movie they also acted in

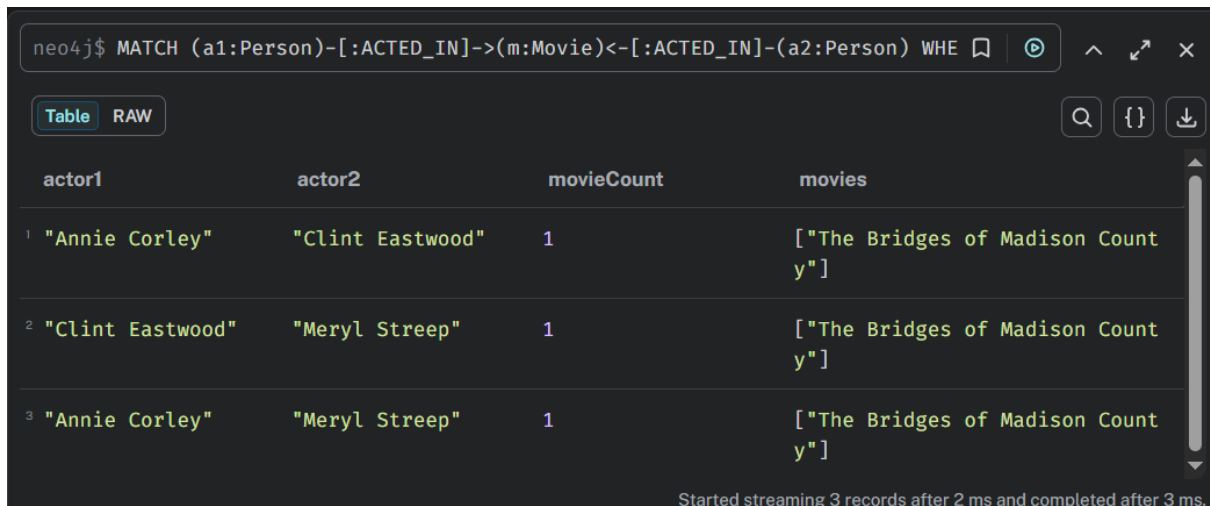
So, this query here finds the people who both acted in and directed the same movie. In this case the query matches a Person node connected to a Movie node through both ACTED_IN and DIRECTED relationships. Finally, this returns actors who also directed the movies they appeared in.

```
MATCH (a:Person)-[:ACTED_IN]->(m:Movie)<-[:DIRECTED]-(a)
RETURN a.name AS actorDirector, collect(DISTINCT m.title) AS movies
ORDER BY actorDirector;
```



Query 3 – actors who acted in multiple movies together

When doing this question the query returned no results, maybe because in the current imported movie graph, no pair of actors acted together in more than one movie. I also performed this table query to check actor pairs and showed that the highest shared movie count was 1



The image shows a screenshot of the Neo4j Cypher query interface. At the top, the query is entered in a text box: `neo4j$ MATCH (a1:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(a2:Person) WHE`. Below the query bar, there are two tabs: "Table" (selected) and "RAW". To the right of the tabs are icons for search, JSON, and download. The results are displayed in a table with four columns: "actor1", "actor2", "movieCount", and "movies". There are three rows of results, all showing a movie count of 1. The first row shows "Annie Corley" and "Clint Eastwood" with the movie "The Bridges of Madison County". The second row shows "Clint Eastwood" and "Meryl Streep" with the same movie. The third row shows "Annie Corley" and "Meryl Streep" with the same movie. At the bottom right, a status message reads: "Started streaming 3 records after 2 ms and completed after 3 ms."

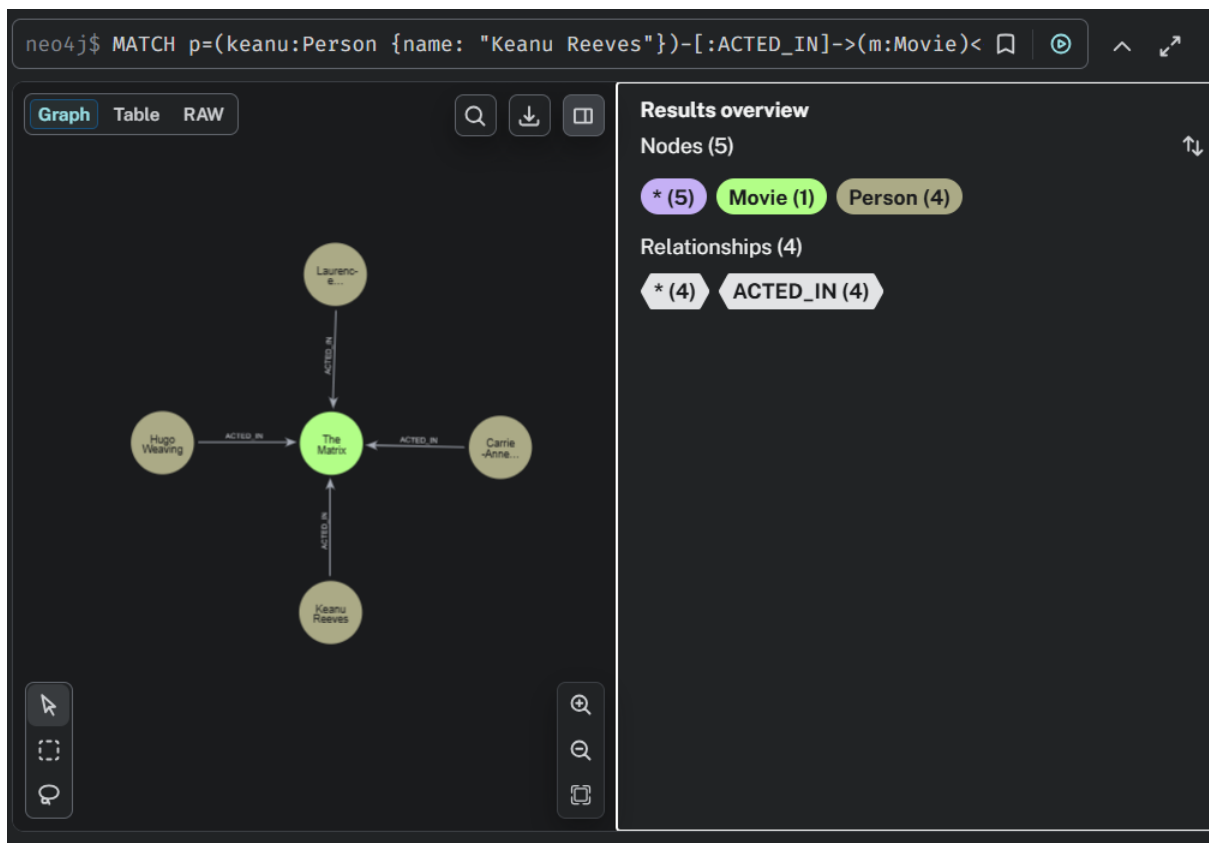
	actor1	actor2	movieCount	movies
¹	"Annie Corley"	"Clint Eastwood"	1	["The Bridges of Madison County"]
²	"Clint Eastwood"	"Meryl Streep"	1	["The Bridges of Madison County"]
³	"Annie Corley"	"Meryl Streep"	1	["The Bridges of Madison County"]

Started streaming 3 records after 2 ms and completed after 3 ms.

Query 4 – Keanu reeves movies and co actors

This query focuses on finding all movies that Keanu Reeves acted in and all lists the co actors who acted in the same movies. The query starts from the Person node for Keanu, follows the ACTED_IN

movieTitle	coActors
"The Matrix"	["Carrie-Anne Moss", "Laurence Fishburne", "Hugo Weaving"]



Query 5 – delete movies with the lowest rating

In this question the query only deletes the movie or movies with the lowest rating. So in order to complete this I first checked which movie had a rating property, then I used `min(m.rating)` to find the lowest rating value. After finding the lowest value, the matching movie nodes were deleted using `DETACH DELETE` which removed the movie and any relationships connected to it

```
1 MATCH (m:Movie)
2 WHERE m.rating IS NOT NULL
3 RETURN m.title AS title, m.rating AS rating
4 ORDER BY rating;
```

Table RAW

title	rating
"The Bridges of Madison County"	7.6

```
1 MATCH (m:Movie)
2 WHERE m.rating IS NOT NULL
3 WITH min(m.rating) AS lowestRating
4 MATCH (m:Movie)
5 WHERE m.rating = lowestRating
6 DETACH DELETE m;
```



Deleted 1 node, deleted 4 relationships

```
neo4j$ MATCH (m:Movie) WHERE m.rating IS NOT NULL RETURN m.title AS title, m
```

No changes, no records

Comp

Part 2 – Personalized learning database

Importing the personalized learning dataset

In order to import the learning csv file, into neo4J desktop, I opened the local instance sever file, access the import folder and placed the csv file in there. Next, I tested the csv file using the LOAD CSV WITH HEADER and returned the first five rows to confirm that the application could read the file correctly.

After confirming that file could be loaded, I cleared the graph to avoid any duplicate nodes and relationships. Next imported the csv data using the previous load command

Moving on, instead of CREATE I used MERGE so that duplicate, student, courses and relationships would not be created if the import query was run more than once.

```
1 LOAD CSV WITH HEADERS FROM
  'file:///personalized_learning_dataset.csv' AS row
2 RETURN row
3 LIMIT 5;
```

row

```
{
  Education_Level:"High School",
  Quiz_Attempts:"4",
  Engagement_Level:"Medium",
  Forum_Participation:"2",
  Assignment_Completion_Rate:"89",
  Feedback_Score:"1",
  Final_Exam_Score:"51",
  Learning_Style:"Visual",
  Age:"15",
  Course_Name:"Machine Learning",
  Student_ID:"S00001",
  Gender:"Female",
  Time_Spent_on_Videos:"171",
  Dropout_Likelihood:"No",
  Quiz_Scores:"67"
```

✓

Created 10,005 nodes, created 10,000 relationships, set 140,005 properties, added 10,005 labels

Query 1

So, this question asks to list all young female students, age below 20 who prefer visual learning style, are in high school and are unlikely to drop out

So here this query list all the young female students who meet the criteria from the question. The query starts from Student nodes and follows the ENROLLED_IN relationship to the course each student is taking. Finally, the WHERE clause filters the students based on gender, age, education level, learning style and dropout likelihood

```
1 MATCH (s:Student)-[e:ENROLLED_IN]->(c:Course)
2 WHERE s.gender = "Female"
3     AND s.age < 20
4     AND s.learningStyle = "Visual"
5     AND s.educationLevel = "High School"
6     AND s.dropoutLikelihood = "No"
7 RETURN
8     s.studentID AS studentID,
9     s.age AS age,
10    s.gender AS gender,
11    s.educationLevel AS educationLevel,
12    s.learningStyle AS learningStyle,
13    c.courseName AS courseName,
14    s.dropoutLikelihood AS dropoutLikelihood
15 ORDER BY s.age, s.studentID;
```

studentID	age	gender	educationLevel	learningStyle	courseName	dropoutLikelihood
"S00001"	15	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S01102"	15	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S01885"	15	"Female"	"High School"	"Visual"	"Web Development"	"No"
"S02528"	15	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S03494"	15	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S03737"	15	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S03825"	15	"Female"	"High School"	"Visual"	"Python Basics"	"No"
"S04678"	15	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S04876"	15	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S05526"	15	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S07115"	15	"Female"	"High School"	"Visual"	"Web Development"	"No"
"S08727"	15	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"

studentID	age	gender	educationLevel	learningStyle	courseName	dropoutLikelihood
"S09371"	15	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S03589"	16	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S03636"	16	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S03818"	16	"Female"	"High School"	"Visual"	"Python Basics"	"No"
"S04660"	16	"Female"	"High School"	"Visual"	"Python Basics"	"No"
"S04751"	16	"Female"	"High School"	"Visual"	"Python Basics"	"No"
"S05624"	16	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S06203"	16	"Female"	"High School"	"Visual"	"Web Development"	"No"
"S06900"	16	"Female"	"High School"	"Visual"	"Python Basics"	"No"
"S06959"	16	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S09476"	16	"Female"	"High School"	"Visual"	"Python Basics"	"No"
"S01133"	17	"Female"	"High School"	"Visual"	"Machine Learning"	"No"

studentID	age	gender	educationLevel	learningStyle	courseName	dropoutLikelihood
"S01237"	17	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S03351"	17	"Female"	"High School"	"Visual"	"Web Development"	"No"
"S04055"	17	"Female"	"High School"	"Visual"	"Web Development"	"No"
"S08726"	17	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S00157"	18	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S00799"	18	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S01271"	18	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S02393"	18	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S02462"	18	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S02572"	18	"Female"	"High School"	"Visual"	"Python Basics"	"No"
"S05130"	18	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S07061"	18	"Female"	"High School"	"Visual"	"Web Development"	"No"

Started streaming 46 records after 460 ms and completed after 509 ms.

studentID	age	gender	educationLevel	learningStyle	courseName	dropoutLikelihood
"S01237"	17	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S03351"	17	"Female"	"High School"	"Visual"	"Web Development"	"No"
"S04055"	17	"Female"	"High School"	"Visual"	"Web Development"	"No"
"S08726"	17	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S00157"	18	"Female"	"High School"	"Visual"	"Machine Learning"	"No"
"S00799"	18	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S01271"	18	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S02393"	18	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S02462"	18	"Female"	"High School"	"Visual"	"Cybersecurity"	"No"
"S02572"	18	"Female"	"High School"	"Visual"	"Python Basics"	"No"
"S05130"	18	"Female"	"High School"	"Visual"	"Data Science"	"No"
"S07061"	18	"Female"	"High School"	"Visual"	"Web Development"	"No"

Started streaming 46 records after 460 ms and completed after 509 ms.

Query 2

For this question all this query does is lists the courses at different education levels where student have high engagement and strong assignment completion. This query matches Student nodes connect to Course nodes through the ENROLLED_IN relationship.

The WHERE also filters the results to only include students with high engagement, it also groups the results by education level and course name. finally the max() function is used to find the highest assignment completion rate for each course at each education level

```
1 MATCH (s:Student)-[e:ENROLLED_IN]->(c:Course)
2 WHERE e.engagementLevel = "High"
3 WITH
4   s.educationLevel AS educationLevel,
5   c.courseName AS courseName,
6   max(e.assignmentCompletionRate) AS highestCompletionRate
7 RETURN
8   educationLevel,
9   courseName,
10  highestCompletionRate
11 ORDER BY highestCompletionRate DESC, educationLevel, courseName;
```

educationLevel	courseName	highestCompletionRate
1 "High School"	"Cybersecurity"	99
2 "High School"	"Data Science"	99
3 "High School"	"Machine Learning"	99
4 "High School"	"Python Basics"	99
5 "High School"	"Web Development"	99
6 "Postgraduate"	"Data Science"	99
7 "Postgraduate"	"Machine Learning"	99
8 "Postgraduate"	"Python Basics"	99
9 "Postgraduate"	"Web Development"	99
10 "Undergraduate"	"Cybersecurity"	99
11 "Undergraduate"	"Data Science"	99
12 "Undergraduate"	"Machine Learning"	99
13 "Undergraduate"	"Python Basics"	99
14 "Undergraduate"	"Web Development"	99
15 "Postgraduate"	"Cybersecurity"	98

Query 3

So, this question ask which course has received the highest feedback rating score

In order to find this out our query must match students connected to courses though the ENROLLED_IN relationship, where the feedback score is stored. Finally we can use the max() function again to find the highest feedback score for each course, then we order the result from highest to lowest, and we limit it to 1 to return only the course with the highest feedback rating.

```
1 MATCH (s:Student)-[e:ENROLLED_IN]->(c:Course)
2 WITH
3   c.courseName AS courseName,
4   max(e.feedbackScore) AS highestFeedbackScore
5 RETURN
6   courseName,
7   highestFeedbackScore
8 ORDER BY highestFeedbackScore DESC
9 LIMIT 1;
```

courseName	highestFeedbackScore
"Machine Learning"	5

Query 4

In this question we need to find out how many students have failed the cybersecurity final exam under 40% due to low engagement.

So, what we have to do is match Student nodes connected to Course nodes through the ENROLLED_IN relationship. Next, we use the WHERE clause which filters the results to only include male students enrolled in cybersecurity, with low engagement and final exam score below 40%. Finally, the count() function is just used to return the total amount of students who meet the conditions.

```
1 MATCH (s:Student)-[e:ENROLLED_IN]->(c:Course)
2 WHERE s.gender = "Male"
3     AND c.courseName = "Cybersecurity"
4     AND e.finalExamScore < 40
5     AND e.engagementLevel = "Low"
6 RETURN count(s) AS failedMaleStudents;
```

failedMaleStudents

Query 5

The final question just ask for how many students are likely to drop out of their courses due to low engagement and list their learning style and educational level

Again, we follow the structure of the last question by matching the Student nodes connected to Course nodes through the ENROLLED_IN relationship. Next , I again used where clause is used to filter the results to include only students with Dropout_Likelihood set to yes and engagement level set to low. The results outputted where grouped by learning style and education level, so the output shows how many students from each group are likely to dropout

```
1 MATCH (s:Student)-[e:ENROLLED_IN]->(c:Course)
2 WHERE s.dropoutLikelihood = "Yes"
3     AND e.engagementLevel = "Low"
4 RETURN
5     s.learningStyle AS learningStyle,
6     s.educationLevel AS educationLevel,
7     count(s) AS numberOfStudents
8 ORDER BY numberOfStudents DESC;
```

learningStyle	educationLevel	numberOfStudents
1 "Auditory"	"Undergraduate"	54
2 "Kinesthetic"	"Undergraduate"	50
3 "Visual"	"Undergraduate"	48
4 "Reading/Writing"	"Undergraduate"	42
5 "Visual"	"High School"	31
6 "Reading/Writing"	"High School"	30
7 "Auditory"	"Postgraduate"	28
8 "Kinesthetic"	"High School"	26
9 "Auditory"	"High School"	25
10 "Kinesthetic"	"Postgraduate"	23
11 "Reading/Writing"	"Postgraduate"	16
12 "Visual"	"Postgraduate"	13